FORMATION

Analyse de données avec Python

Jour 2, matin
Agrégation, dates et automatisation de plusieurs fichiers

Cours et TP3 en alternance, sur neuf mois de ventes

Le déroulé de la matinée

1. **Données du jour** — puis TP3, partie 0
2. **Plusieurs fichiers** — puis TP3, partie 1
3. **Jointures** — puis TP3, partie 2
4. **Pause**
5. **Agréger** — puis TP3, partie 3
6. **Dates** — puis TP3, partie 4
7. **Indicateurs et export** — puis TP3, partie 5 et synthèse

9h00

début

10h30

pause

12h30

fin

Chaque section de cours est courte et suivie aussitôt de la partie du TP3 qui lui correspond.

01

Les données du jour

Retour sur le TP2 | Ventes mensuelles | Tables de référence

1 Des données en plusieurs morceaux

En entreprise, les données arrivent rarement dans un seul fichier.

Fichier	Contenu	Clé
`ventes/ventes_2025_01.csv` à `_09.csv`	une ligne par commande : date, région, produit, quantité, prix payé	`id_commande`
`produits.csv`	le catalogue : nom, catégorie, prix catalogue	`id_produit`
`regions.csv`	les régions : nom complet, objectif de chiffre d'affaires mensuel	`code_region`

Les ventes ne contiennent que des codes (`P03`, `IDF`). Pour des indicateurs lisibles, il faut assembler les fichiers puis les joindre aux tables de référence.

1 Pratique : TP3, partie 0

Mise en place, dans exercices/tp03-indicateurs/tp03.ipynb

À faire

Activer l'environnement du TP1, ou en recréer un à partir de son `requirements.txt`
Lancer `jupyter lab` depuis le dossier du TP, puis ouvrir le notebook
Parcourir le dossier `ressources` : un fichier de ventes, puis les deux tables de référence
Exécuter la première cellule : imports et `DOSSIER`

Dans le notebook

Repère	Rôle
Résultat attendu	à comparer avec ce qu'on obtient
`# Vérification`	contrôle avec `assert` : affiche `OK`, sinon une `AssertionError` explique le problème
Pour aller plus loin	facultatif, pour qui termine une partie en avance

Les tables construites au début (`fichiers`, `ventes`, `df`) servent jusqu'à la fin du TP.

02

Parcourir et empiler des fichiers

pathlib | glob | Boucle et concat | Contrôles

2 pathlib : manipuler des chemins

```
from pathlib import Path

DOSSIER = Path("ressources")
fichier = DOSSIER / "ventes" / "ventes_2025_01.csv"

fichier.name      # 'ventes_2025_01.csv'
fichier.stem      # 'ventes_2025_01'
fichier.suffix    # '.csv'
fichier.parent    # ressources/ventes
fichier.exists()  # True
```

L'opérateur `/` construit un chemin valable sur tous les systèmes

- Préférable à la concaténation de chaînes, source d'erreurs sous Windows

`pd.read_csv` accepte directement un `Path`

Le mois dans le nom du fichier

```
fichier.stem.split("_")
# ['ventes', '2025', '01']
int(fichier.stem.split("_")[-1])
# 1
```

2 glob : lister les fichiers

```
fichiers = sorted((DOSSIER / "ventes").glob("*.csv"))
len(fichiers)    # 9

# le premier trimestre : un caractère entre 1 et 3
(DOSSIER / "ventes").glob("ventes_2025_0[1-3].csv")

# récursif : 11 fichiers, sous-dossiers compris
DOSSIER.rglob("*.csv")
```

Pattern	Correspond à
`*`	n'importe quelle suite de caractères
`?`	un seul caractère
`[1-3]`	un caractère de la liste

`glob` ne garantit aucun ordre : on trie avec `sorted` pour un traitement reproductible.

2 Lire chaque fichier, assembler une seule fois

```
morceaux = []
for fichier in fichiers:
    morceau = pd.read_csv(fichier, parse_dates=["date"])
    morceau["fichier"] = fichier.name
    morceaux.append(morceau)

ventes = pd.concat(morceaux, ignore_index=True)
ventes.shape    # (2182, 8)
```

La colonne `fichier` garde l'**origine** de chaque ligne
`ignore_index=True` renumérote les lignes de 0 à 2181
Sans lui, l'index contient des doublons : `ventes.loc[0]` renvoie 9 lignes, une par fichier
Un seul `concat` à la fin : dans la boucle, il recopierait tout à chaque tour

2 Plus court : compréhension et fonction

En une compréhension

```
ventes_bis = pd.concat(
    [pd.read_csv(f, parse_dates=["date"])
       .assign(fichier=f.name)
     for f in fichiers],
    ignore_index=True,
)
ventes_bis.equals(ventes)    # True
```

Avec une fonction

```
def charger_ventes(fichier):
    morceau = pd.read_csv(fichier,
                          parse_dates=["date"])
    morceau["fichier"] = fichier.name
    return morceau

ventes = pd.concat(
    [charger_ventes(f) for f in fichiers],
    ignore_index=True,
)
```

`equals` vérifie que deux tables sont identiques : mêmes colonnes, mêmes valeurs, même index. La fonction se teste sur un seul fichier avant d'être appliquée à tous ; le jour 3, on la déplacera dans un module réutilisable.

2 concat aligne les colonnes par leur nom

ventes_2025_01.csv

```
id_commande,date,code_region,id_produit,...
```

ventes_2025_05.csv, exporté avec un autre outil

```
date,id_commande,id_produit,code_region,...
```

Corriger un nom avant d'empiler, avec rename vu hier

```
morceau = morceau.rename(columns={"Date": "date"})
```

Ordre différent : **aucun problème**, les colonnes sont associées par nom
Nom différent (`Date` au lieu de `date`) : une **colonne en plus**, remplie de `NaN`, sans aucun message d'erreur

D'où l'importance de contrôler le résultat après l'assemblage.

2 Contrôler l'assemblage

```
# avant : les lignes de chaque fichier
{f.name: len(pd.read_csv(f)) for f in fichiers}
# {'ventes_2025_01.csv': 226, ...}

# après : les lignes par fichier
ventes["fichier"].value_counts().sort_index()
# ventes_2025_01.csv    226
# ventes_2025_02.csv    229
# ...

ventes.columns
ventes.isna().sum()
ventes["id_commande"].is_unique
```

Contrôle	Détecte
lignes par fichier	un fichier vide ou tronqué
liste des colonnes	une colonne renommée
valeurs manquantes	des colonnes mal alignées
unicité de la clé	un mois exporté deux fois

Les comptes avant et après l'assemblage doivent être identiques.

2 Pratique : TP3, partie 1

Parcourir un dossier et empiler les fichiers

Questions	Au programme
1.1 à 1.3	construire un chemin, tirer le mois du nom du fichier, lister avec `glob` et `rglob`
1.4 et 1.5	compter les lignes de chaque fichier, comparer les colonnes du fichier de mai
1.6 à 1.8	la fonction `charger_ventes`, un seul `concat`, l'effet de `ignore_index`
1.9	contrôler l'assemblage

Pour aller plus loin : 1.10 (`assign` et `equals`) et 1.11 (une colonne `Date` mal nommée).

03

Joindre des tables

merge | Types de jointure | Pièges

3 merge : associer des lignes par une clé

ventes

id_commande	id_produit	quantite
10001	P03	2
10002	P10	15
10003	P03	1

produits

id_produit	produit	categorie
P03	Écran 24 pouces	Informatique
P10	Bloc-notes	Papeterie

```
ventes_produits = ventes.merge(produits, on="id_produit", how="left")

# clés de noms différents : les deux colonnes clés sont conservées
ventes.merge(produits, left_on="id_produit", right_on="code_produit")
```

3 Les types de jointure

	Lignes conservées
`inner`	clés présentes des deux côtés (par défaut)
`left`	toutes les lignes de gauche
`right`	toutes les lignes de droite
`outer`	toutes les lignes des deux tables

Les colonnes sans correspondance sont remplies de `NaN`.

Sur les 2182 ventes

2165

avec `inner`

2182

avec `left`

17 ventes disparaissent avec la jointure par défaut, sans aucun message.

3 inner, left et outer sur un petit exemple

commandes

id_commande	code_client
1	C1
2	C2
3	C4

clients

code_client	nom
C1	Durand
C2	Martin
C5	Leroy

Résultat avec `how="outer"`

id_commande	code_client	nom
1.0	C1	Durand
2.0	C2	Martin
3.0	C4	NaN
NaN	C5	Leroy

```
len(commandes.merge(clients, on="code_client"))                  # inner : 2
len(commandes.merge(clients, on="code_client", how="left"))      # left : 3, sans nom pour C4
len(commandes.merge(clients, on="code_client", how="outer"))     # outer : 4, avec C5 sans commande
```

Une colonne d'entiers ne peut pas contenir `NaN` : dans la jointure `outer`, `id_commande` devient décimale (`1.0`, `2.0`…).

3 Trouver les lignes sans correspondance

```
controle = ventes.merge(produits, on="id_produit",
                        how="left", indicator=True)

controle["_merge"].value_counts()
# both          2165
# left_only       17
# right_only       0

controle.loc[controle["_merge"] == "left_only",
             "id_produit"].unique()
# ['P17']
```

`indicator=True` ajoute une colonne `_merge`
`left_only` : présent seulement à gauche
`right_only` : présent seulement à droite, par exemple un client sans commande avec `how="outer"`

`P17` est vendu depuis août mais absent du catalogue : une information métier à remonter, pas une erreur à masquer.

3 Les pièges des jointures

1

Des types différents

Une clé `int` d'un côté et `str` de l'autre : `ValueError`. Les variantes d'écriture (`p03`, un espace) empêchent aussi la correspondance.

2

Des clés en double

Un produit présent deux fois dans le catalogue duplique toutes ses ventes. Les totaux sont faux, sans message d'erreur.

3

Des colonnes homonymes

Deux colonnes `libelle` deviennent `libelle_x` et `libelle_y`. Avec `suffixes=("_vente", "_produit")` : `libelle_vente` et `libelle_produit`.

Le nettoyage d'hier (espaces, casse, types) est ce qui rend les jointures fiables.

3 Se protéger des clés en double

Vérifier

```
# comparer avant et après
len(ventes), len(ventes_produits)

# faire vérifier la relation
ventes.merge(produits, on="id_produit",
             how="left",
             validate="many_to_one")
# MergeError si une clé est en double
# dans produits
```

Corriger la table de référence

```
# après avoir vérifié la bonne ligne
produits = produits.drop_duplicates(
    subset="id_produit"
)
```

`drop_duplicates(subset=...)` garde la première occurrence de chaque clé. On relance ensuite la jointure.

3 Enchaîner les jointures

```
regions = pd.read_csv(DOSSIER / "regions.csv")

df = (
    ventes
    .merge(produits, on="id_produit",
           how="left", validate="many_to_one")
    .merge(regions, on="code_region",
           how="left", validate="many_to_one")
)
df.shape    # (2182, 13)
```

Garder les ventes hors catalogue

```
df[["produit", "categorie"]] = (
    df[["produit", "categorie"]]
    .fillna("Non référencé")
)
```

Supprimer les ventes de `P17` fausserait le chiffre d'affaires : un libellé explicite les garde visibles dans tous les indicateurs.

`join` donne le même résultat qu'un `merge` en `how="left"`, avec l'index de la table de droite comme clé : `equals` le confirme.

3 Pratique : TP3, partie 2

Joindre des tables, d'abord sur de petits exemples, puis sur les ventes

Questions	Au programme
2.1 à 2.4	petites tables : `inner`, `left`, `outer`, `indicator`, une clé en double, `left_on` et `suffixes`
2.5 à 2.7	les vraies données : clés uniques, 2165 lignes contre 2182, le produit `P17`
2.8 et 2.9	la table `df` et le libellé `Non référencé`

Pour aller plus loin : 2.10, la même jointure avec `join`, vérifiée avec `equals`.

04

Agréger

groupby et agg | transform | pivot_table et crosstab

4 Les agrégations simples

```
df["montant"] = df["prix_unitaire"] * df["quantite"]

df["montant"].sum()            # 409501.59
df["montant"].mean()           # 187.67
df["montant"].median()         # 104.70
df["id_commande"].count()      # 2182
df["id_produit"].nunique()     # 17
len(df)                        # 2182
```

`sum` : le chiffre d'affaires total
`mean` : le panier moyen, bien au-dessus du montant **médian**
`count` : les valeurs non manquantes, `len` : les lignes
`nunique` : 17 produits différents vendus, `P17` compris

4 groupby : découper, calculer, rassembler

1 **Découper**

une table par région

2 **Calculer**

la somme des montants de chaque table

3 **Rassembler**

un résultat par région

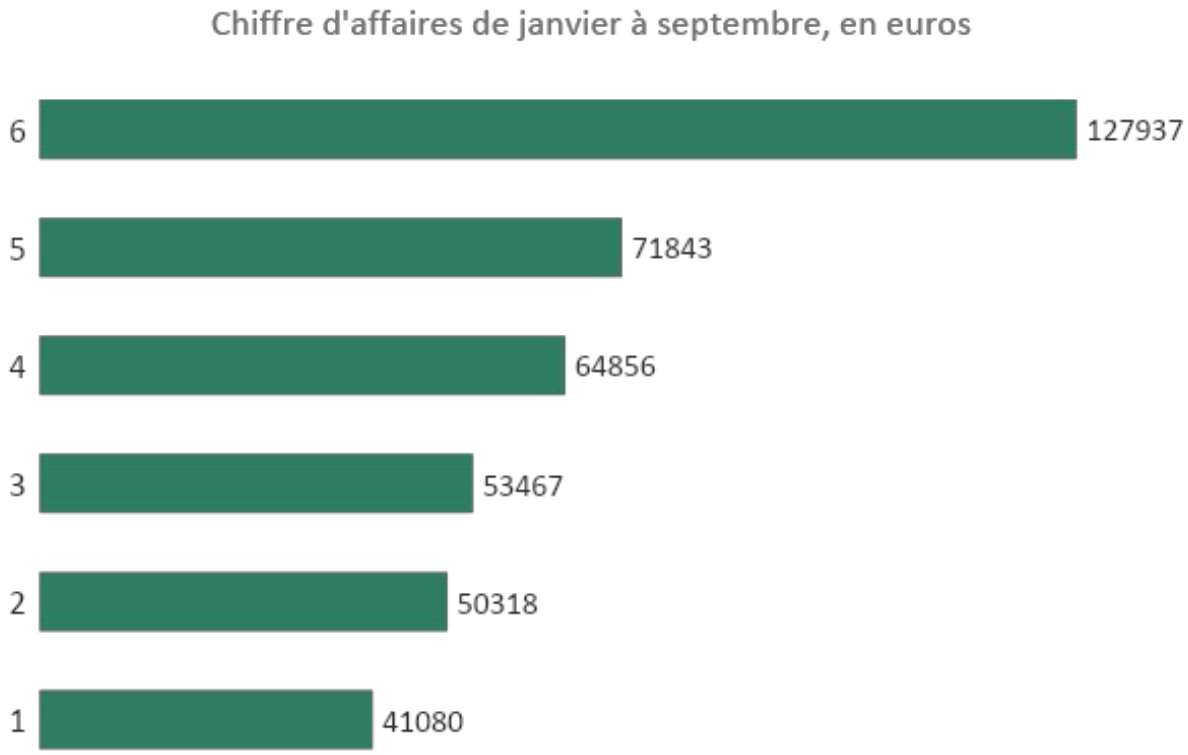
```
df.groupby("region")["montant"].sum()     # pour chaque région, la somme des montants
```

4 Le chiffre d'affaires par région

```
df.groupby("region")["montant"].sum() \
  .sort_values(ascending=False)

# plusieurs clés
df.groupby(["region", "categorie"])["montant"].sum()

# clés en colonnes
df.groupby(["region", "categorie"],
           as_index=False)["montant"].sum()
```



4 Attention aux clés manquantes

La table controle de la section 3.3 : les ventes de P17 n'ont pas de catégorie

```
controle["montant"] = controle["prix_unitaire"] * controle["quantite"]

controle.groupby("categorie")["montant"].sum().sum()                  # 406827.97 : P17 a disparu
controle.groupby("categorie", dropna=False)["montant"].sum().sum()    # 409501.59 : avec une ligne NaN
```

Pour vérifier qu'une agrégation n'a rien perdu : total des groupes égal au total général.

409 502

chiffre d'affaires total

406 828

somme des catégories

D'où le libellé `Non référencé` donné aux ventes de `P17` dans `df`.

4 agg : plusieurs indicateurs nommés

```
indicateurs = df.groupby("region").agg(
    ca=("montant", "sum"),
    nb_commandes=("id_commande", "count"),
    panier_moyen=("montant", "mean"),
    nb_produits=("id_produit", "nunique"),
)
```

Chaque argument se lit : `nom=("colonne", "fonction")`.

region	ca	nb_commandes	panier_moyen
Auvergne-Rhône-Alpes	64856	378	171.58
Bretagne	41080	209	196.55
Hauts-de-France	53467	320	167.09
Nouvelle-Aquitaine	71843	313	229.53
Occitanie	50318	288	174.72
Île-de-France	127937	674	189.82

4 Les fonctions d'agrégation

Fonction	Calcule
`sum`, `mean`, `median`	somme, moyenne, médiane
`min`, `max`, `std`	extrêmes, écart-type
`count`	valeurs non manquantes
`size`	lignes du groupe
`nunique`	valeurs distinctes non manquantes
`first`, `last`	première, dernière valeur

size contre count

```
mini = pd.DataFrame({
    "region": ["IDF", "IDF", "BRE"],
    "montant": [120.0, None, 210.0]})

mini.groupby("region").agg(
    nb_lignes=("montant", "size"),
    nb_montants=("montant", "count"),
)
#         nb_lignes  nb_montants
# region
# BRE             1            1
# IDF             2            1
```

4 transform : un résultat par ligne

```
df["ca_region"] = (
    df.groupby("region")["montant"]
      .transform("sum")
)
df["part_ca_region"] = df["montant"] / df["ca_region"]
```

`agg` renvoie une ligne par **groupe**
`transform` renvoie une valeur par **ligne d'origine**

- Chaque vente reçoit le chiffre d'affaires de sa région, puis sa part dans ce total
- Les parts d'une même région totalisent 1

4 Les meilleurs éléments de chaque groupe

Les plus grandes valeurs

```
(df.groupby("produit")["montant"].sum()
   .nlargest(5))
# Écran 24 pouces         66480.75
# Chaise ergonomique      62088.15
# Clavier mécanique       40695.25
# Bureau assis-debout     38403.75
# Casque audio            35064.15
```

`nlargest`, vu hier, fonctionne aussi sur une Series ; `nsmallest` donne les plus petites valeurs.

Le meilleur produit de chaque région

```
ca_region_produit = df.groupby(
    ["region", "produit"], as_index=False
)["montant"].sum()

(
    ca_region_produit
    .sort_values("montant", ascending=False)
    .groupby("region")
    .head(1)
)
# Île-de-France       Écran 24 pouces     24267.60
# Nouvelle-Aquitaine  Chaise ergonomique  16446.45
```

Après un `groupby`, `head(n)` garde les `n` premières lignes de chaque groupe : c'est le tri préalable qui en fait les `n` meilleures.

4 pivot_table : le tableau croisé

```
df["mois"] = df["date"].dt.to_period("M")

pd.pivot_table(
    df,
    index="mois",
    columns="categorie",
    values="montant",
    aggfunc="sum",
    fill_value=0,
    margins=True,
)
```

mois	Bureau	Informatique	Non réf.	Papeterie	Électro.	All
2025-01	14838	17431	0	6667	4539	43475
2025-02	10014	16029	0	6002	5784	37830
…						
2025-08	7580	13600	1296	2691	2107	27274
2025-09	17513	28474	1378	6840	3869	58074
All	124548	185275	2674	59181	37824	409502

Un `groupby` sur deux clés, dont la seconde passe en colonnes. `margins=True` ajoute les totaux `All`, `fill_value=0` remplace les combinaisons absentes par 0.

4 crosstab : compter des combinaisons

```
pd.crosstab(df["region"], df["client_fidele"])

# avec les totaux
pd.crosstab(df["region"], df["client_fidele"],
            margins=True)

# en proportions par ligne
pd.crosstab(df["region"], df["client_fidele"],
            normalize="index")
```

- Compte les occurrences de chaque combinaison de deux colonnes

`margins=True` ajoute les totaux, `normalize="index"` donne des proportions par ligne

- Ici : entre 33 % et 39 % de clients fidèles selon la région

Avec `values` et `aggfunc` : la part de chaque catégorie par région

```
pd.crosstab(df["region"], df["categorie"], values=df["montant"],
            aggfunc="sum", normalize="index")
```

4 Pratique : TP3, partie 3

Agréger la table des ventes enrichies

Questions	Au programme
3.1 à 3.5	montant, agrégations simples, `groupby`, clés manquantes dans `controle`
3.6 et 3.7	agrégation nommée ; `size`, `count` et `nunique` sur une petite table
3.8 à 3.10	`transform`, `nlargest`, le meilleur produit de chaque région
3.11 et 3.12	`pivot_table` par mois et par région, `crosstab` de la fidélité

Pour aller plus loin : 3.13 (`crosstab` avec `values`) et 3.14 (le taux de remise).

05

Manipuler les dates

Accesseur .dt | resample | shift et pct_change | rolling

5 Calculer et extraire avec .dt

```
df["date"].min(), df["date"].max()
# du 1er janvier au 30 septembre
(df["date"].max() - df["date"].min()).days    # 272
df["date"].nunique()    # 270 jours avec une vente
df["date"] + pd.Timedelta(days=30)

df["annee"] = df["date"].dt.year
df["mois_num"] = df["date"].dt.month
df["jour_semaine"] = df["date"].dt.dayofweek
df["mois"] = df["date"].dt.to_period("M")
```

Attribut	Exemple
`.dt.year`	2025
`.dt.month`	3
`.dt.dayofweek`	0 pour lundi
`.dt.to_period("M")`	2025-03

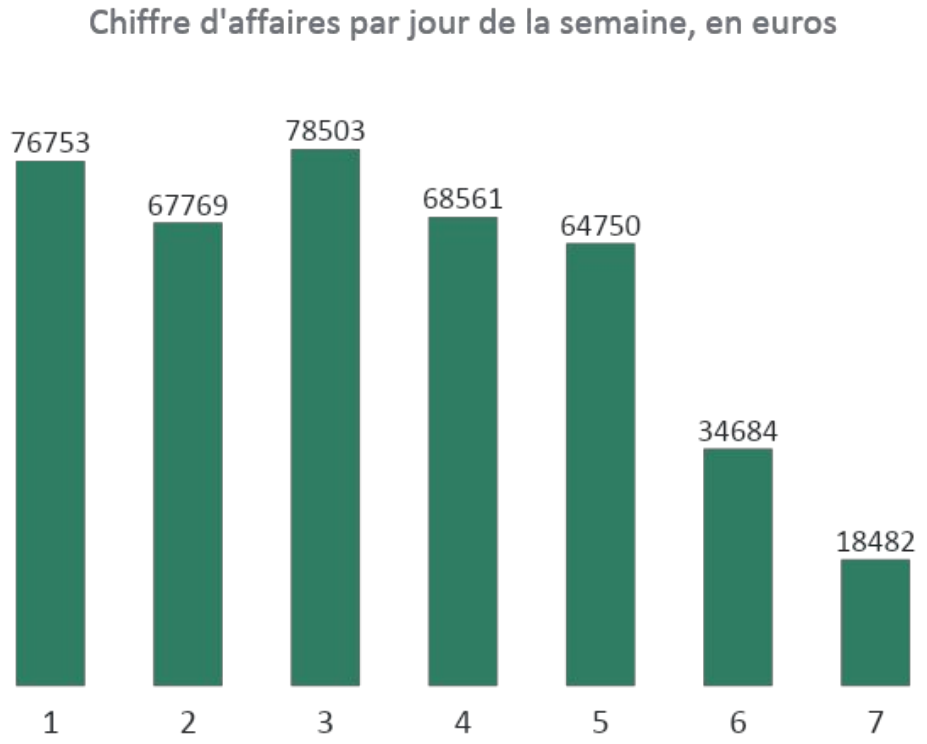
`to_period("M")` fait une excellente clé de regroupement mensuel. `.dt.to_timestamp()` fait l'inverse : la date du premier jour, utile avant un export.

5 Les jours de la semaine en français

```
jours = {0: "lundi", 1: "mardi",
         2: "mercredi", 3: "jeudi",
         4: "vendredi", 5: "samedi",
         6: "dimanche"}
df["jour"] = df["date"].dt.dayofweek.map(jours)

# dans l'ordre de la semaine
(df.groupby("jour")["montant"].sum()
   .reindex(list(jours.values())))
```

`groupby` trie les jours par ordre alphabétique : dimanche, jeudi, lundi...
`reindex`, vu hier, les remet dans l'ordre de la semaine.



5 Filtrer sur des dates

Avec des conditions

```
df[df["date"] >= "2025-07-01"]

# bornes incluses : 307 ventes
df[df["date"].between("2025-07-01",
                      "2025-08-31")]
df[df["date"].dt.month.isin([7, 8])]  # idem

# les 30 derniers jours : 319 ventes
df[df["date"] > df["date"].max()
              - pd.Timedelta(days=30)]
```

Avec un index de dates

```
par_date = df.set_index("date").sort_index()

par_date.loc["2025-07"]             # juillet
par_date.loc["2025-07":"2025-08"]   # l'été
par_date.loc["2025-08-15"]          # un jour
```

Une date partielle sélectionne toute la période : un mois, une année, une journée. `set_index` renvoie une nouvelle table : `df` garde sa colonne `date`.

5 resample : regrouper par période

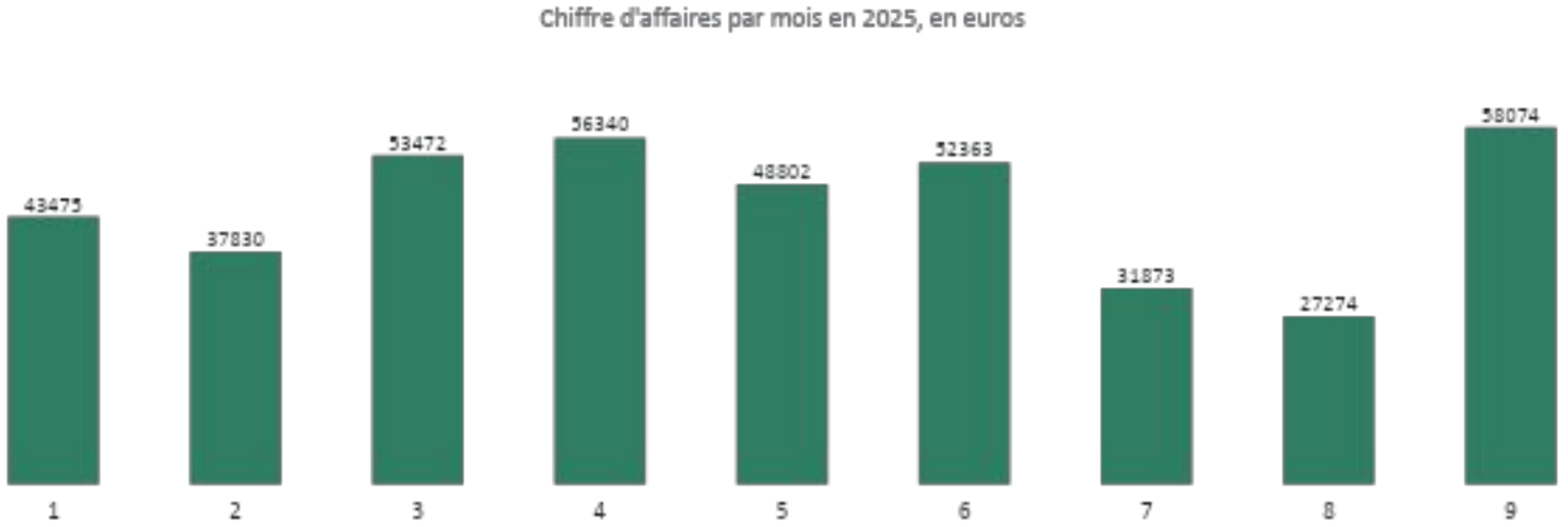
```
df.resample("MS", on="date")["montant"].sum()
# 2025-01-01    43475
# 2025-02-01    37830
# 2025-03-01    53472
# ...
```

Un `groupby` sur le temps, qui crée aussi les périodes sans données : indispensable pour une moyenne mobile juste.

Code	Période
`D`	jour
`W`	semaine, finie le dimanche
`MS`, `ME`	mois : début, fin
`QS`, `QE`	trimestre
`YS`, `YE`	année

Avec pandas 3, `"M"` n'est plus accepté : il faut `"MS"` ou `"ME"`.

5 Le chiffre d'affaires mensuel



Un creux en juillet et août, un pic à la rentrée : un indicateur mensuel se lit toujours avec la saisonnalité en tête.

5 Attention aux semaines incomplètes

```
df.resample("W", on="date")["montant"].sum()
# 2025-01-05     4688
# 2025-01-12    11482
# ...
# 2025-09-28    11343
# 2025-10-05     4595
```

Chaque semaine porte le label de son **dernier jour**, un dimanche

- La première ne contient que 5 jours de données, du mercredi 1er au dimanche 5 janvier
- La dernière n'en contient que 2 : les 29 et 30 septembre

Les totaux de ces semaines incomplètes, plus faibles, ne se comparent pas à ceux des semaines complètes.

5 Comparer à la période précédente

```
ca_mensuel = df.resample("MS", on="date")["montant"].sum()

ca_mensuel.shift(1)         # la valeur du mois précédent : la série décalée d'une ligne
ca_mensuel.diff()           # l'écart avec le mois précédent, en euros
ca_mensuel.pct_change()     # la variation relative : (mois - mois précédent) / mois précédent
```

mois				
2025-01	43475	NaN	NaN	NaN
…				
2025-07	31873	52363	-20490	-0.391
2025-08	27274	31873	-4599	-0.144
2025-09	58074	27274	30800	1.129

Janvier n'a pas de mois précédent : `NaN`
`pct_change` renvoie une proportion : on la multiplie par 100 pour un pourcentage

- Juillet : -39,1 %, septembre : +112,9 %

5 rolling, expanding et cumuls

```
ca_jour = df.resample("D", on="date")["montant"].sum()

ca_jour.rolling(7).mean()      # 7 derniers jours
ca_jour.expanding().sum()      # cumul
ca_jour.cumsum()               # même résultat

# à quelle date atteint-on 200 000 euros ?
cumul = ca_jour.cumsum()
cumul[cumul >= 200_000].index[0]
# Timestamp('2025-05-08 00:00:00')
```

`rolling(7)` : moyenne des 7 jours qui se terminent ce jour-là
Les 6 premiers jours donnent `NaN`, faute d'historique

- Une fenêtre de 7 jours efface l'effet du jour de la semaine

`expanding` : une fenêtre qui s'agrandit, pour les cumuls

5 Pratique : TP3, partie 4

Manipuler les dates

Questions	Au programme
4.1 et 4.2	période couverte, jours distincts, jour de la semaine en français
4.3 et 4.4	filtres de dates, sélection par date partielle
4.5 et 4.6	les jours dans l'ordre avec `reindex`, le chiffre d'affaires mensuel
4.7 et 4.8	l'évolution avec `shift` et `pct_change`, les semaines incomplètes
4.9 et 4.10	jours sans vente, moyenne mobile, seuil de 200 000 euros

Pour aller plus loin : 4.11, les dates du maximum et du minimum de la moyenne mobile.

06

Construire et exporter les indicateurs

Mois et région | Taux de réalisation | Export Parquet

6 Une ligne par mois et par région

```
indicateurs = df.groupby(["mois", "region"], as_index=False).agg(
    ca=("montant", "sum"),
    nb_commandes=("id_commande", "count"),
    panier_moyen=("montant", "mean"),
    quantite=("quantite", "sum"),
    objectif_mensuel=("objectif_mensuel", "first"),
)
indicateurs.shape    # (54, 7) : 9 mois x 6 régions
```

`first` prend la première valeur de chaque groupe

- Elle convient à une valeur identique sur tout le groupe, comme l'objectif mensuel d'une région

Contrôle : la somme de `ca` égale le chiffre d'affaires total, celle de `nb_commandes` le nombre de ventes.

6 Des indicateurs calculés

```
indicateurs["taux_realisation"] = indicateurs["ca"] / indicateurs["objectif_mensuel"]

indicateurs.nlargest(3, "taux_realisation")     # les 3 meilleures combinaisons de mois et de
région
indicateurs.nsmallest(3, "taux_realisation")    # les 3 moins bonnes

# un calcul intermédiaire, sans modifier la table
indicateurs.assign(atteint=indicateurs["taux_realisation"] >=
1).groupby("region")["atteint"].sum()
# nombre de mois où l'objectif est atteint : 6 en Île-de-France, 2 en Occitanie...
```

Un ratio se calcule **après** l'agrégation, à partir des totaux

`assign` travaille sur une copie : la table garde exactement les colonnes prévues

L'évolution dans chaque région

```
indicateurs.groupby("region")["ca"].pct_change()
```

Chaque mois est comparé au mois précédent de la même région, pas à la ligne précédente du tableau.

6 Exporter, puis relire

```
# le type période est propre à pandas : on le convertit en date du premier jour du mois
indicateurs_export = indicateurs.assign(mois=indicateurs["mois"].dt.to_timestamp())
indicateurs_export.to_parquet("indicateurs_mois_region.parquet", index=False)

# relire le fichier produit pour le contrôler
relu = pd.read_parquet("indicateurs_mois_region.parquet")
relu.shape     # (54, 8)
relu.dtypes    # mois en datetime64, region en str, les indicateurs en nombres
```

Tous les outils ne savent pas lire le type période : `.dt.to_timestamp()` en fait une date
`assign` convertit sur une copie : `indicateurs` garde sa colonne `mois` en période

- On termine toujours en relisant le fichier : forme, colonnes et types

6 Choisir les colonnes exportées

```
colonnes_ventes = [
    "id_commande", "date", "code_region", "id_produit", "quantite", "prix_unitaire",
    "client_fidele", "fichier", "produit", "categorie", "prix_catalogue", "region",
    "objectif_mensuel", "montant", "jour",
]
df[colonnes_ventes].to_parquet("ventes_consolidees.parquet", index=False)
```

Une table exportée est un contrat : le tableau de bord de cet après-midi attend des colonnes précises, dans un ordre précis.

La liste explicite écarte les colonnes de travail créées en cours de route (`mois`, `part_ca_region`…).

6 Pratique : TP3, partie 5

La table d'indicateurs par mois et par région, puis la synthèse

Questions	Au programme
5.1	la table `indicateurs` : 54 lignes, 7 colonnes dans l'ordre imposé
5.2 et 5.3	taux de réalisation, mois où l'objectif est atteint, bilan des neuf mois
5.4 et 5.5	export des deux fichiers Parquet, puis relecture
Synthèse	trois ou quatre constats tirés des indicateurs, pour la direction commerciale

Pour aller plus loin : 5.6 (`nlargest` et `nsmallest`) et 5.7 (l'évolution dans chaque région).

7 TP3 : construction d'indicateurs consolidés

Dans le notebook exercices/tp03-indicateurs/tp03.ipynb, complété au fil de la matinée

1

Assembler

Parties 1 et 2 : parcourir le dossier, empiler les 9 fichiers, joindre produits et régions en contrôlant chaque étape.

2

Calculer

Parties 3 et 4 : agrégations, tableaux croisés, dates, évolutions d'un mois sur l'autre et moyennes mobiles.

3

Livrer

Partie 5 : `indicateurs_mois_region.parquet` et `ventes_consolidees.parquet`, pour le TP4 de cet après-midi.

Chaque partie commence sur de petits exemples, puis passe aux vraies données. Les cellules de vérification contrôlent les réponses avec `assert`.

À retenir

`pathlib` pour les chemins, `glob` et `sorted` pour lister
Une liste de DataFrames, puis un seul `concat`

- Garder le fichier d'origine, compter les lignes par fichier

`concat` aligne par nom : un nom mal écrit crée une colonne
`merge` en `left` pour enrichir, `indicator=True` pour vérifier
Lignes avant et après, `validate` contre les doublons

- Un libellé explicite plutôt que de supprimer

`groupby` ignore les clés manquantes : comparer les totaux

`agg` nommée ; `size`, `count` et `nunique` à distinguer
`transform` par ligne, tri puis `head(n)` par groupe
`pivot_table` pour croiser, `crosstab` pour compter
`.dt`, `to_period("M")`, `reindex` pour l'ordre des jours
`resample` crée les périodes vides ; attention aux semaines incomplètes
`shift` et `pct_change` comparent à la période précédente
`rolling` lisse, `cumsum` cumule
Exporter : `to_timestamp()`, colonnes choisies, relecture